Figma → 代码：组件复用与设计还原度

痛点 4 的验证 Demo · 仓库 luckylucky0001/figma-fe · 2026-10-20 · 交付物：CLI `figmafe`（Go）+ Skill `figma-to-code` + H5 / Android Demo + 教程

结论

- **流程跑通、可量化**：「先盘点资产 → 建立映射 → 出复用方案 → 实现 → 审计 + 截图对比 → 修正 → 沉淀规则」在 H5（React）和 Android（Compose，兼容 compileSdk 32）两端都实现为一个命令 `figmafe verify`。它输出复用率、违规、像素差异和节点级偏差，可以直接接入 CI。
- **实测**：13 次对照运行里，C 组（完整 Skill）在 3/3 次 H5 运行中达标。Android 端剩下的 2 处严重偏差都出在**组件库内部**，A 组和 C 组相同。工具能把「页面写错了」和「组件本身不符合设计」区分开。
- **如实说明**：在这个 demo 里，强模型的基线（A 组）也能找到组件库，复用率达到 100%。差别体现在「规范和可追溯」上：A 组 7/8 次自己造了一个规范外的会跳动的 CountDown 组件，C 组 0/5 次；A 组 2/8 次挂了 Figma 节点 id，C 组 5/5 次。token 漂移、变体缺失这类资产问题，只有 C 组的扫描和映射检查系统性地找了出来。
- **换成小模型（Haiku）**：结论一样。两端的复用率和还原度都和大模型持平，C 组同样 0 次自造组件。Skill 的价值不依赖模型大小：把「找组件、定方案、验收」做成确定性的工具步骤，模型只负责按方案写代码和修正。
- **建议**：在真实项目中，先拿一个典型页面按教程接入，用 `figmafe verify` 量出现状（这是 demo 无法替代的一步），再推广到 PR CI。

1. 方案：把 Proposal 落成可执行的工具链

① 导出
免费 Figma 插件（Plugin API）导出节点、变量、2 倍图；不用付费的 MCP 和 Code Connect

② 盘点资产
`scan`：两端组件、props、变体、示例、使用次数、token、存量坏实践

③ 映射
`map suggest/check`：Figma ↔ H5 / Android；输出缺口、变体不支持、token 漂移

④ 方案闸门
`plan`：节点 → 组件 → 可粘贴的代码；人确认后才写

⑤ 验证闭环
`verify`：静态审计 + 截图 + 像素 / 节点比对，最多修 3 轮

⑥ 沉淀
`rules.md` 和组件问题清单，下个页面直接受益

Proposal 要求	落地方式	业界依据
梳理 Figma 与组件的对应关系	`component-map.yaml`（两端实现、变体 → props、文本图层 → 属性、约束、缺口决策）；`map suggest` 按名称、同义词和变体结构打分给出候选	Figma Code Connect 的多平台映射思路；Brad Frost 的 Interface Inventory
位置、示例、约束放进 Context	`catalog` 从资产清单和映射生成组件手册，Skill 通过渐进式加载读取	Storybook Component Manifest；Mística 随 npm 包附带 `llms.md`
开发前识别复用、确认方案	`plan` 输出复用方案和待决项，Skill 规定必须先确认	Figma MCP 推荐的「先查映射再生成」
开发后检查组件、截图对照	`audit`（原生元素、直接用 Material、Legacy 组件、写死颜色、疑似自造组件）+ `diff`（像素 + 文本、组件节点级偏差）	pixelmatch、Roborazzi、Compose semantics、设计系统 lint
偏差反馈给 Agent 修正	`report.md` 每条偏差带文件行号或 Figma 节点 id、期望值和实际值；退出码 3 表示未达标，Agent 照着修	Agent Skills：确定性的步骤交给脚本
先验证一个页面，再整理成规则	订单详情页两端都验证过；规则合并进 `rules.md`，组件问题汇总到 `component-issues.md`	—

2. 对照实验

同一份 Figma（订单详情，含 21 个组件实例、1 个缺口组件 CountDown）、同一套组件库、同样的任务描述。每次运行都在独立的工作区里完成，最后用同一版 `figmafe verify` 统一打分。C 组的方案闸门在实验中视为自动通过。

组	端	结果	复用率	违规	像素差异	节点偏差 严重/轻微	自造 CountDown	Figma id	耗时 / token
A 基线	H5	PASS	100%	0	0.65%	0 / 5	是	0	90s / 89k
A 基线	H5	PASS	100%	0	0.65%	0 / 5	是	0	104s / 99k
A 基线 · npm 包	H5	PASS	100%	0	0.65%	0 / 5	是	0	104s / 95k
A 基线 · 只给截图	H5	PASS	100%	0	0.65%	0 / 5	是	0	275s / 106k
A 基线 · Haiku	H5	PASS	100%	0	0.65%	0 / 5	是	0	96s / 93k
C 完整 Skill	H5	PASS	100%	0	0.65%	0 / 5	否	27	212s / 123k
C 完整 Skill · npm 包	H5	PASS	100%	0	0.65%	0 / 5	否	27	186s / 102k
C 完整 Skill · Haiku	H5	PASS	100%	0	0.65%	0 / 5	否	24	194s / 144k
A 基线	Android	FAIL	100%	0	4.00%	2 / 6	是	28	143s / 105k
A 基线 · 只给截图	Android	FAIL	100%	0	4.00%	2 / 6	是	0	245s / 114k
A 基线 · Haiku	Android	FAIL	100%	0	4.00%	2 / 6	否	27	182s / 121k
C 完整 Skill	Android	FAIL	100%	0	4.00%	2 / 6	否	27	301s / 129k
C 完整 Skill · Haiku	Android	FAIL	100%	0	4.00%	2 / 6	否	27	260s / 163k

复用率 = 方案要求的组件中实际用到的比例（静态统计，并用运行时的 `data-ds` 和 semantics 标记复核）。阈值：复用率 ≥95%，违规 0，像素差异 H5 ≤3%、Android ≤5%，节点级严重偏差 0。

verify 能抓到什么（教程第 8 步，故意放了 3 处问题）



读结果

- **复用率**：只给截图、组件库是 npm 包、换成 Haiku，基线都能做到 100%，所以在这个小仓库里，复用率区分不了 A、C 两组。真实项目复用差的原因（仓库大、命名对不上、两套组件、设计稿没有组件化）需要拿真实项目来量。
- **规范**：A 组 7/8 次在页面里自造了 CountDown，其中 6 次还带了计时逻辑（设计稿里没有这个需求，组件库也没有）。时间一长，这类组件就会变成重复资产。C 组 5 次都按缺口方案处理：用静态文本实现，并登记组件需求。
- **可追溯**：C 组每个实例都挂了 Figma 节点 id，偏差可以精确定位到节点。
- **成本**：C 组平均耗时是 A 组的 1.5 倍，token 是 1.3 倍，换来的是可以复现的验证结果和资产问题清单。
- **闭环有效**：第 8 步故意放进的 3 处问题被全部抓到（3 处违规 + 8 处严重偏差），修正后一次通过。

3. 资产层面的发现（Phase 0 / 1 的副产品）

发现	来源	处理
Android `color.warning` 是 #FF8800，Figma 是 #FF8F1F（token 漂移）	`map check`	组件库修正
Android `DsBadge` 缺 Solid 变体；`LegacyCellView` 不支持副标题	`map check` / `scan`	补变体或在 Figma 上标注
CountDown：Figma 有、两端都没有	`map check` / `plan` 待决项	登记需求，本期按静态文本实现
Android 同时存在 Compose（Ds*）和 XML View（Legacy*）两套资产	`map suggest` 给出并列候选	决策：新页面只用 Compose
H5 Primary 按钮宽 2px、Tag 贴顶、ProductCard 内容下移 4px；Android DsPriceText 高度、DsGoodsCard 规格行重叠	`verify` 节点级偏差（A、C 组一致）	见 `docs/component-issues.md`
存量 XML 页面里有原生 `<Button>`，并写死了 #FF4D1A、14sp	`scan` 存量问题（基线）	迁移

4. 工具校准（实验中发现、已修复）

- 文本位置改为「左对齐、居中、右对齐任一吻合即算一致」：Figma 文本框常常是定宽，代码里多是按内容收缩，原来按左上角比对会误报（C 组 Agent 发现）。
- 文本优先匹配纯文本节点，避免匹配到同时带文本的组件根节点（例如 DsTopBar 的标题；C 组 Android Agent 发现）。
- Android 像素阈值从 4% 调到 5%：实现完全正确时，Skia 和 Chromium 的字形渲染本底约为 4.0%。

5. 局限（诚实说明）

- 没有真实 Figma 文件，环境也访问不了 Figma：设计数据由独立手写的设计源生成（`figmafe fixture`），格式和插件导出一致；插件代码没有在真实 Figma 里跑过。
- 组件库是按规范自建的小型库（两端各 10 个组件），不是你们的真实资产；仓库小，有利于基线 Agent 找到组件。
- 样本量小（每种条件 1～2 次），结论只能说明方向，不能当作统计显著的结果。
- Android：组件用 Compose 实现，只用 Compose 1.2 已有的 API（对应 compileSdk 32 / minSdk 23），扫描同时支持 XML View。本环境装不了 Android SDK，所以用 Compose Desktop 在 JVM 上渲染（API 相同），不是真机，字体渲染存在本底差异（已通过阈值吸收）。真实项目把渲染器换成 Roborazzi 或 Paparazzi 即可。

6. 接入真实项目（下一步）

#	动作	产出
1	用插件导出组件库和一个典型页面（例如开户或交易的某一页）	`design/figma-export/`
2	修改 `figmafe.yaml`，指向真实的 H5 组件库（源码或 npm 包的 `.d.ts`）和 Android 组件库（Compose 或 XML View）	资产清单和存量问题基线
3	`map suggest` → 组件 owner 确认 → `map check`	映射文件、缺口 / 漂移清单
4	A / C 对照各跑 3 次，量出真实项目的复用率和偏差	真实的 before / after 数据
5	`verify` 接入 PR CI；`rules.md` 随页面增多持续积累	可持续的规则和门禁

详细步骤和每一步的截图见 `docs/tutorial/README.md`；数据见 `experiments/`（每次运行的报告、截图、源码）；重新生成：`python3 experiments/summarize.py && python3 docs/tools/build_report.py && node docs/tools/shoot.mjs html docs/report.html docs/report.pdf`。